

Prompt — Patrimônio 1M

Cole este documento inteiro como instrução para o agente de desenvolvimento (Claude Code, Cursor, v0, Lovable, Bolt etc.).

Trechos marcados com **[DECISÃO]** são escolhas que você pode mudar antes de enviar.

0. PAPEL E FORMA DE TRABALHO

Você é um time sênior de produto e engenharia (tech lead full-stack, designer de produto fintech e engenheiro de segurança) construindo um SaaS de planejamento de investimentos para pessoas físicas no Brasil, com arquitetura pronta para expansão internacional.

Regras de trabalho:

- Antes de escrever código**, entregue um plano curto: estrutura de pastas, schema do banco, lista de rotas, matriz Free x Premium e as premissas de cálculo. Depois execute fase por fase.
- Não invente dados de mercado**. Nenhum preço, dividendo, indicador ou rentabilidade histórica pode ser fixado no código ou no seed.
- Nada de funcionalidade de fachada**. Todo botão, link e menu visível precisa funcionar. O que for de fases futuras fica atrás de *feature flag* desligada e **não aparece** na interface (nem como "em breve", salvo na página de planos).
- Quando houver ambiguidade**, escolha a opção mais conservadora do ponto de vista regulatório e de segurança, registre a decisão em `docs/DECISIONS.md` e siga em frente.
- Ao final de cada fase**, rode lint, typecheck e testes e relate com honestidade o que passou, o que falhou e o que não foi testado.

1. PRODUTO

Nome provisório: Patrimônio 1M

Subtítulo: Seu plano inteligente para construir patrimônio.

Conceito: Transforme seus aportes em um plano estruturado para construir patrimônio.

Metáfora central: um *GPS financeiro*. O usuário define o destino (a meta) e o app mostra onde ele está, quanto falta, em que ritmo está indo e o que precisaria mudar para chegar antes.

Público inicial: pessoa física brasileira, 18 anos ou mais, iniciante ou intermediária, que investe todo mês e quer um plano, não dicas de ações.

Diferencial: não é só uma calculadora. Junta, num ciclo contínuo:

Meta → Aporte → Carteira-alvo → Juros compostos → Acompanhamento → Rebalanceamento → Proventos
→ Cenários → Educação

1.1 Princípio inegociável: simulação, não promessa

- Toda projeção é uma **SIMULAÇÃO** baseada em **premissas editáveis**, que aparecem sempre ao lado do resultado ("Premissas: 9% a.a. nominal, inflação 4% a.a., aporte no fim do mês, valores antes de impostos e taxas").
- O app **não promete** rentabilidade, lucro, renda nem enriquecimento.
- Rentabilidade passada não garante rentabilidade futura. Investimentos têm risco, inclusive de perda do capital.
- Carteiras-modelo e sinais são **ferramentas educacionais e de planejamento**, não recomendação individual de investimento.

1.2 Enquadramento regulatório (Brasil)

O produto **não** é consultoria (Resolução CVM 19/2021) nem análise de valores mobiliários (Resolução CVM 20/2021), e o questionário **não** é uma *suitability* (Resolução CVM 30/2021). Por isso:

- O perfil é chamado de "**perfil de risco autodeclarado**", nunca de "suitability" ou "perfil de investidor oficial".
- Carteiras-modelo são definidas por **classe de ativo**. O app não sugere tickers específicos para o usuário comprar.
- Tickers aparecem apenas quando o próprio usuário os cadastra na carteira dele ou como exemplos no catálogo, sem juízo de compra ou venda.
- O "Motor de Oportunidades" (seção 13) só avalia **critérios definidos pelo próprio usuário** e fica fora do MVP até revisão jurídica.
- Crie `docs/LEGAL_REVIEW.md` listando todos os pontos que precisam de validação por advogado antes do lançamento: textos legais, carteiras-modelo, oportunidades, IA e nome/marca (consulta ao INPI).

1.3 Linguagem

Proibido	Usar no lugar
"lucro garantido", "renda garantida", "retorno certo"	"resultado simulado", "estimativa sob as premissas"
"ganhe X%", "fique rico", "enriqueça"	"com rentabilidade hipotética de X% a.a."
"COMPRA AGORA", "oportunidade imperdível", "você vai ganhar"	"Este ativo atende aos critérios definidos no seu modelo."
"recomendamos", "o melhor investimento"	"carteira-modelo educacional", "exemplo de alocação"
"você vai atingir R\$ 1 milhão em X anos"	"sob estas premissas, a simulação atinge R\$ 1 milhão em X anos"

Centralize os textos em arquivos de mensagens (i18n). Crie um teste que procura as expressões proibidas nos arquivos de mensagens e nas páginas e falha se encontrar alguma.

2. IDENTIDADE VISUAL E UX

- **Estética:** fintech premium, sóbria, minimalista e confiável. Referências de tom: bancos digitais e gestoras modernas. Nada de visual de cassino, day trade, foguetes, cifrões dourados ou "to the moon".
 - **Paleta:** base neutra (grafite/off-white), uma cor primária de confiança (ex.: verde-petróleo ou azul profundo) e uma de destaque discreta. Verde e vermelho só para variação, nunca como chamariz. Tema claro e escuro.
 - **Tipografia:** sans-serif legível com números tabulares (`font-variant-numeric: tabular-nums`) em todas as tabelas e valores.
 - **Mobile-first:** pensar primeiro em 360 px de largura e depois expandir para tablet, notebook e desktop. Gráficos legíveis e com tooltip no toque.
 - **Acessibilidade:** WCAG 2.1 AA (contraste, foco visível, navegação por teclado, `aria-label` nos gráficos e tabela alternativa para cada gráfico).
 - **Gamificação profissional:** progresso, marcos e conquistas discretos. Sem confete exagerado, sem sons e sem rankings entre usuários.
 - **Regra dos 2 minutos:** um usuário novo precisa entender onde está e quanto falta em menos de 2 minutos após o onboarding.
 - **Formatação:** pt-BR (R\$ 1.234,56; 9,5% a.a.; datas dd/mm/aaaa), fuso `America/Sao_Paulo` . Formatação sempre via `Intl` , nunca concatenando strings.
-

3. ESCOPO POR FASES

FASE 1 — MVP (construir agora, 100% funcional)

1. Landing page
2. Cadastro, login (e-mail/senha e Google), confirmação de e-mail, recuperação de senha, logout e exclusão de conta
3. Onboarding com diagnóstico inicial
4. Meta financeira (uma no Free, várias no Premium)
5. Calculadora rumo à meta, com cenários e simulação de juros compostos
6. Dashboard principal
7. Carteiras-modelo por perfil (por classe de ativo), editáveis
8. Registro manual de **movimentações** (aporte, resgate, provento) e **atualização manual de saldo** por classe
9. Calendário e acompanhamento de aportes
10. Modo "Meta de R\$ 1 milhão" com marcos
11. Comparador de cenários de aporte
12. Rebalanceamento por classe de ativo, priorizando novos aportes
13. Planos Free e Premium, paywalls e Stripe (Checkout, Customer Portal e webhooks)
14. Banco de dados com RLS
15. LGPD: Termos de Uso, Política de Privacidade, Política de Cookies, consentimentos, exportação e exclusão de dados
16. Painel administrativo básico
17. Central educativa com conteúdo inicial (mínimo de 6 artigos originais)

FASE 2 — Arquitetura pronta agora, implementação depois (atrás de feature flag)

- Integração `MarketDataProvider` com provedor licenciado
- Catálogo de ativos com dados de mercado e páginas `/ativos`, `/fiis`, `/fiagro`, `/exterior`, `/cripto` e `/tesouro`
- Rebalanceamento por ativo individual
- Painel completo de dividendos e proventos
- Alertas por e-mail e notificações no app
- Relatórios mensais em PDF
- Importação de carteira por CSV, preço médio e operações

FASE 3 — Futuro

- Assistente de IA
- Motor de Oportunidades (após revisão jurídica)
- Planos Pro e Família, afiliados, cursos, e-books e conteúdo premium
- Integrações oficiais com corretoras e bancos (Open Finance), só por API autorizada

Importante: no MVP não há dados de mercado. O patrimônio vem dos **saldos informados pelo usuário** ("Atualizar saldo") somados às movimentações registradas. Deixe isso explícito na interface ("Valores informados por você em dd/mm/aaaa").

4. ONBOARDING

Passo a passo curto (barra de progresso, no máximo uma ou duas perguntas por tela, com opção de voltar). Todas as respostas podem ser editadas depois em `/configuracoes`.

#	Pergunta	Tipo / validação
1	Quanto você já tem investido hoje?	moeda, ≥ 0
2	Quanto consegue investir por mês?	moeda, ≥ 0
3	Pretende aumentar seus aportes a cada ano? Em quanto?	%, de 0 a 50, padrão 0
4	Qual seu objetivo?	R\$ 100 mil / 250 mil / 500 mil / 1 milhão (pré-selecionado) / 2 milhões / Personalizado
5	Em quanto tempo quer chegar lá?	anos, de 1 a 60, ou "não sei — calcule para mim"
6	Como você reage a oscilações? (3 perguntas situacionais simples)	gera o perfil Conservador, Moderado, Arrojado ou Agressivo, que o usuário pode ajustar
7	Sua faixa de idade	faixas (18-24, 25-34, ...). Não pedir data de nascimento: minimização de dados
8	Você tem reserva de emergência?	Sim / Parcial / Não
9	Que parte do seu patrimônio total já está investida?	faixas

#	Pergunta	Tipo / validação
10	Tem dívidas com juros altos (cartão, cheque especial)?	Sim / Não
11	Quer, no futuro, viver de renda dos investimentos?	Sim / Não / Ainda não sei
12	Quais classes você quer na sua carteira?	múltipla escolha: Renda fixa, Tesouro Direto, Ações BR, FII's, Fiagro, ETFs internacionais, Criptoativos

Diagnóstico inicial (tela de resultado e primeira entrega de valor, antes de qualquer paywall):

- "Você tem R\$ X. Sua meta é R\$ Y. Faltam R\$ Z."
- Projeção no cenário do perfil: "Sob estas premissas, a simulação atinge a meta em N anos e M meses."
- Aporte necessário para o prazo escolhido.
- Carteira-modelo sugerida **por classe** para o perfil autodeclarado, com aviso educacional.
- **Alertas de prioridade, em tom educativo e sem julgamento:**
- dívida cara → "Quitar dívidas com juros altos costuma vir antes de investir."
- sem reserva → "Montar uma reserva de emergência costuma ser o primeiro passo."
- prazo curto com perfil agressivo → explicar o risco de volatilidade em prazos curtos.
- CTA: "Ver meu painel".

5. MOTOR DE CÁLCULO (núcleo do produto)

5.1 Regras gerais

- Todos os cálculos ficam em **um único pacote de domínio puro** (`packages/finance-core` ou `src/domain/finance`), sem dependência de UI ou banco, com 100% de cobertura de testes.
- Os mesmos cálculos rodam no servidor (fonte da verdade, em relatórios e na persistência) e podem rodar no cliente apenas para pré-visualização interativa, **importando o mesmo módulo**. Proibido duplicar fórmula.
- Use aritmética decimal (`decimal.js` ou `big.js`). Proibido `number/float` para dinheiro.
- Banco: dinheiro em `NUMERIC(20,2)` , quantidades em `NUMERIC(28,10)` (cripto), taxas em `NUMERIC(12,8)` .
- Arredondamento: cálculos intermediários com precisão total; arredondar **só na exibição e na persistência final**, em 2 casas, `ROUND_HALF_UP` . Documente a regra.
- Toda simulação salva grava o `assumption_set_id` (versão das premissas usadas), para ser reproduzível depois que o admin mudar os padrões.

5.2 Convenções (documentar em `docs/FINANCE.md` e mostrar na UI em "Como calculamos")

- Taxa mensal equivalente: $i_m = (1 + i_a)^{(1/12)} - 1$. Nunca $i_a / 12$.
- Aporte no **fim de cada mês** (postecipado) por padrão, configurável.
- Crescimento do aporte aplicado **uma vez por ano**, a cada 12 meses.
- Taxa real (Fisher): $r = (1 + i_{nominal}) / (1 + inflação) - 1$.

- Valores **antes de impostos, taxas e custos**. Campo opcional de "custos anuais estimados (%)", subtraído da taxa. Aviso sobre IR (tabela regressiva na renda fixa, isenções específicas e regras de FII/ FIAGRO) apenas como texto educativo. Não calcular IR no MVP.
- **Meta nominal x meta real**: toggle "Mostrar em valores de hoje (descontada a inflação)". Explicar que R\$ 1 milhão daqui a 20 anos compra menos que R\$ 1 milhão hoje.

5.3 Funções obrigatórias

- `projectMonthly(inputs)` → série mês a mês { mês, saldoInicial, aporte, rendimento, saldoFinal, totalAportado, rendimentoAcumulado }
- `futureValue(pv, pmt, i_m, n, growth, timing)`
- `requiredMonthlyContribution(pv, meta, i_m, n, growth)` : fórmula fechada quando `growth = 0` ; busca binária com tolerância de R\$ 0,01 quando houver crescimento
- `timeToGoal(pv, pmt, i_m, meta, growth)` : em meses; retorna "não atinge em 100 anos" quando for o caso
- `compoundInterestBreakdown(serie)` : separa total aportado, rendimento total e **juros sobre juros** (= rendimento total – rendimento que existiria em capitalização simples sobre o mesmo fluxo)
- `crossoverPoints(serie)` : (a) primeiro mês em que o rendimento do mês supera o aporte do mês; (b) primeiro mês em que o rendimento acumulado supera o total aportado
- `realValue(valorNominal, inflação, meses)`
- `modifiedDietzReturn(saldoInicial, saldoFinal, fluxosDados)` : rentabilidade do período da carteira real do usuário
- `allocationDrift(alvo, atual)` e `rebalanceWithContribution(alvo, atual, novoAporte)`

5.4 Testes com valores esperados (mínimo)

Caso	Esperado
PV 0, aporte R\$ 1.000/mês, 1% a.m., 12 meses, fim do mês	R\$ 12.682,50
PV R\$ 10.000, sem aporte, 10% a.a., 10 anos	R\$ 25.937,42
12% a.a. → taxa mensal	0,948879% a.m.
Taxa 0%	FV = PV + soma dos aportes (sem divisão por zero)
<code>requiredMonthlyContribution</code> aplicado e reprojetoado	atinge a meta com erro $\leq$ R\$ 0,01
Meta já atingida ($PV \geq meta$)	tempo = 0 e aporte necessário = 0
Meta inatingível (aporte 0, taxa 0, $PV < meta$)	retorno explícito "não atinge", sem loop infinito

Inclua também testes de propriedade (ex.: mais aporte nunca leva mais tempo) e testes de arredondamento, conversão de períodos, rebalanceamento e Dietz.

Os exemplos do tipo "Ano 10: aportes $\approx$ rendimentos" **não devem ser fixados**. Os marcos do gráfico vêm sempre de `crossoverPoints` com as premissas atuais.

6. FUNCIONALIDADES DO MVP

6.1 Dashboard (/dashboard)

Topo, em linguagem natural (o "GPS"):

Você tem **R\$ X**. Seu objetivo é **R\$ Y**. Faltam **R\$ Z**.

Com seu aporte atual de R\$ A/mês, **sob as premissas do cenário Moderado**, a simulação atinge a meta em **N anos**.

Para chegar em **P anos**, o aporte precisaria ser de **R\$ B/mês**.

Cards: Patrimônio atual · Meta · Quanto falta · % da meta · Aporte do mês (meta × realizado) · Total aportado · Proventos registrados · Rentabilidade do período (Modified Dietz, com tooltip explicando) · Previsão de chegada.

Gráficos:

- Evolução patrimonial real (histórico de saldos) + projeção tracejada a partir de hoje.
- Trilha de marcos: R\$ 1 mil → 10 mil → 50 mil → 100 mil → 250 mil → 500 mil → 750 mil → 1 milhão, indicando atingido, atual e próximo.
- Alocação atual × alocação-alvo por classe.

Todo card mostra a data da última atualização de saldo.

6.2 Calculadora rumo à meta (/simulador)

- **Entradas:** patrimônio inicial, aporte mensal, crescimento anual do aporte, rentabilidade anual, inflação, custos anuais, prazo, meta, momento do aporte.
- **Saídas:** patrimônio projetado (nominal e em valores de hoje), total aportado, rendimento acumulado, juros sobre juros, tempo até a meta e aporte necessário para o prazo.
- **Cenários lado a lado:** Conservador, Moderado e Agressivo. **[DECISÃO]** Padrões **nominais** de 6%, 9% e 12% a.a. com inflação de 4% a.a., todos editáveis pelo admin (versionados) e sobrescrevíveis pelo usuário em sua simulação.
- Rótulo fixo: "Premissas hipotéticas para simulação. Não são previsão nem promessa."
- Botão "Salvar simulação" (Premium: ilimitado; Free: 1).

6.3 Simulação de juros compostos

- Gráfico de área empilhada: **dinheiro aportado** × **rendimento acumulado**, com linha de patrimônio total.
- Marcadores calculados por `crossoverPoints`, com legenda: "A partir daqui, o rendimento do mês supera o seu aporte."
- Tabela anual alternativa (acessibilidade e exportação CSV no Premium).

6.4 Modo "Meta de R\$ 1 milhão" (/meta)

- 🎯 Meta, patrimônio, % percorrido ("Você já percorreu 3,75% do caminho"), próximo marco e quanto falta para ele.

- Marcos: R\$ 10 mil, 50 mil, 100 mil, 250 mil, 500 mil, 750 mil e 1 milhão, com data em que cada um foi atingido (registrada automaticamente).
- Estimativa de data de cada marco futuro **sob as premissas**.
- Tom profissional: conquista discreta, sem euforia.

6.5 Comparador de cenários (/simulador/comparar)

Matriz de **aportes** (padrão: R\$ 500, 1.000, 2.000 e 3.000, editáveis) × **cenários de rentabilidade**, mostrando o tempo até a meta em cada célula e um gráfico de linhas. No Free, aparece só a linha do aporte atual.

6.6 Carteiras-modelo (/carteira)

- Quatro modelos **por classe de ativo**: Conservadora, Moderada, Arrojada e Agressiva. Percentuais iniciais definidos pelo admin no seed. **[DECISÃO]** Exemplo do Moderado/Arrojado: 25% renda fixa/Tesouro, 25% ações BR, 15% FIIs, 15% exterior/ETFs, 10% cripto, 5% Fiagro e 5% reserva tática.
- O usuário pode copiar um modelo e editar os percentuais (validação: soma = 100%, cada classe entre 0 e 100%).
- Aviso fixo: "Carteira-modelo educacional para fins de planejamento. Não é recomendação de investimento individual."
- Carteira do usuário: saldo por classe (e, opcionalmente, por ativo que ele mesmo cadastrar como texto livre ou a partir do catálogo).

6.7 Aportes e movimentações (/aportes)

- Calendário mensal: meta do mês, aportado, falta e status (em dia, parcial, atrasado).
- Registro: data, tipo (aporte, resgate, provento), valor, classe, ativo (opcional) e observação. Editar e excluir, com log de auditoria.
- Histórico com filtros e totais por mês e por ano.

6.8 Rebalanceamento por classe

- Tabela alvo × atual × desvio em pontos percentuais. Faixa de tolerância configurável (padrão ± 5 p.p.).
- Mensagens neutras: "FIIs estão 6 p.p. abaixo da alocação-alvo." / "Ações BR estão 3 p.p. acima da alocação-alvo."
- **Prioridade**: "Como distribuir o próximo aporte de R\$ X para se aproximar do alvo" (`rebalanceWithContribution`). Venda só aparece como informação secundária ("rebalancear vendendo exigiria..."), com aviso sobre possíveis impostos e custos.

6.9 Central educativa (/aprender)

Categorias: Investimentos, Ações, FIIs, Fiagro, Tesouro, Renda fixa, ETFs, Cripto, Juros compostos, Diversificação, Risco, Rebalanceamento e Planejamento financeiro. Artigos em MDX ou no banco, gerenciáveis pelo admin, com data de revisão. Conteúdo original, sem dados de mercado fixos e com links para fontes oficiais (Tesouro Direto, B3, CVM, Banco Central).

7. PLANOS E MONETIZAÇÃO

7.1 Matriz Free × Premium (centralizada em entitlements, nunca espalhada em ifs)

Recurso	Free	Premium
Metas	1	ilimitadas
Calculadora e 3 cenários	✓	✓
Simulações salvas	1	ilimitadas
Comparador de aportes	só o aporte atual	matriz completa
Valores em termos reais (inflação)	✓	✓
Carteira-modelo	visualizar	copiar, editar, várias carteiras
Registro de aportes e movimentações	últimos 3 meses	ilimitado
Dashboard	essencial	completo
Rebalanceamento	resumo	detalhado + distribuição do aporte
Histórico e exportação CSV	—	✓
Fase 2+ (alertas, PDF, dividendos, IA...)	—	incluídos quando lançados

[DECISÃO] Preços iniciais: **R\$ 29,90/mês** ou **R\$ 299,00/ano** (economia de ~17%), com período de teste configurável (padrão de 7 dias).

7.2 Paywalls

- O usuário só encontra o paywall **depois** de ver o próprio diagnóstico e a primeira simulação.
- Paywall elegante: prévia desfocada do recurso real **com os dados do usuário**, uma frase de valor e CTA "Experimentar Premium". Nada de bloquear a tela inteira.
- Funil: Landing → Cadastro → Onboarding → Diagnóstico → Dashboard → recurso Premium → paywall → Stripe Checkout → retorno → Dashboard Premium.

7.3 Arquitetura para planos futuros

Tabelas `plans`, `prices` e `entitlements` genéricas, suportando Pro, Família (assentos), anual, cupons e afiliados sem mudança de schema. Não implementar esses planos agora.

8. STRIPE

- **Stripe Checkout** (modo `subscription`) e **Stripe Customer Portal** para cancelar, trocar de plano, atualizar cartão e baixar faturas. Nenhum dado de cartão passa pelo nosso servidor nem é guardado no banco.
- Moeda BRL. **[DECISÃO]** Formas de pagamento: cartão no MVP. Avalie a disponibilidade atual de Pix e boleto para **assinaturas recorrentes** no Stripe Brasil e documente em `DECISIONS.md`; não presuma que existe.

- Cupons e códigos promocionais via Stripe (`allow_promotion_codes`). O admin cria cupons pelo painel, que chama a API do Stripe.
 - **Preços são imutáveis no Stripe:** "alterar preço" no admin cria um **novo Price** e o define como padrão para novas assinaturas. Assinantes atuais mantêm o preço antigo, salvo migração explícita. Mostre isso claramente no admin.
 - **Webhooks** (rota com verificação de assinatura via raw body):
`checkout.session.completed`, `customer.subscription.created`, `customer.subscription.updated`, `customer.subscription.deleted`, `customer.subscription.trial_will_end`, `invoice.paid`, `invoice.payment_failed`, `invoice.payment_action_required`.
 - **Idempotência:** gravar `event.id` em `webhook_events` e ignorar duplicados; tolerar eventos fora de ordem relendo a assinatura na API do Stripe antes de atualizar.
 - **Acesso Premium** derivado do status real: `active` e `trialing` → Premium; `past_due` → Premium durante período de carência configurável (padrão 7 dias) com aviso no app; `canceled`, `unpaid`, `incomplete_expired` → Free. Cancelamento ao fim do período mantém o acesso até `current_period_end`.
 - **Downgrade não apaga dados:** o que excede o Free fica somente leitura.
 - E-mails transacionais: boas-vindas, trial terminando, pagamento falhou e assinatura cancelada.
 - **Nota fiscal:** deixe um ponto de integração (`InvoiceIssuer`) para emissão de NFS-e por um provedor brasileiro e registre a pendência em `LEGAL_REVIEW.md`.
 - Script `scripts/stripe-setup` que cria produtos e preços no modo de teste, e instruções para usar a Stripe CLI (`stripe listen`) localmente.
-

9. AUTENTICAÇÃO E SEGURANÇA

9.1 Autenticação

Cadastro com e-mail e senha (mínimo de 10 caracteres, checagem contra senhas vazadas quando o provedor suportar), login com Google, confirmação de e-mail obrigatória, recuperação de senha, logout de todas as sessões e exclusão de conta. Aceite de Termos e Política de Privacidade no cadastro, com versão e data registradas. Idade mínima de 18 anos (autodeclaração).

9.2 Segurança

- **RLS em todas as tabelas com dados de usuário** (`user_id = auth.uid()`), com testes automatizados que tentam ler e escrever dados de outro usuário e **devem falhar**.
- `service_role` e chaves secretas só no servidor. Nada sensível com prefixo `NEXT_PUBLIC_`.
- Validação com Zod em toda entrada (formulários, server actions, rotas de API e webhooks).
- Rate limiting em login, cadastro, recuperação de senha, webhooks e APIs públicas.
- Cabeçalhos: CSP restritiva, HSTS, `X-Frame-Options / frame-ancestors`, `Referrer-Policy`. Cookies `HttpOnly`, `Secure` e `SameSite`. Proteção CSRF em mutações fora do fluxo padrão do framework.
- Sem SQL concatenado; apenas client tipado ou queries parametrizadas.
- `audit_logs` para ações sensíveis: login, mudança de senha, exportação, exclusão, ações de admin e mudanças de assinatura.

- Observabilidade com Sentry (ou equivalente), **sem enviar valores financeiros ou PII** nos eventos.
- Backups automáticos do Postgres com retenção documentada e procedimento de restauração testado e descrito em `docs/RUNBOOK.md`.
- Dependências auditadas no CI.

9.3 Admin e privacidade

- O papel de admin vem de uma claim no servidor, nunca do cliente. Rotas `/admin` protegidas no middleware e no servidor.
- Por padrão, o admin vê **métricas agregadas** e dados de conta (e-mail, plano, status, datas), **não** saldos, metas e movimentações individuais.
- Acesso a dados financeiros de um usuário só via "modo suporte", com justificativa obrigatória, tempo limitado e registro em `audit_logs`.

10. LGPD

- Identificação do **controlador** e do **encarregado (DPO)** com canal de contato em `/privacidade`.
- Base legal de cada tratamento documentada: execução de contrato (conta e planos), legítimo interesse (segurança e prevenção a fraude), consentimento (e-mails de marketing e cookies não essenciais).
- Dados financeiros informados pelo usuário não são "dados sensíveis" pelo art. 5º, II, mas são tratados como **confidenciais**: criptografia em trânsito e em repouso e acesso mínimo.
- **Minimização**: sem CPF, sem data de nascimento completa e sem dados bancários no MVP.
- Banner de cookies com opção de recusar os não essenciais. Analytics só após consentimento.
- **Central de privacidade** em `/configuracoes/privacidade`: baixar meus dados (JSON + CSV), revogar consentimentos e excluir conta.
- **Exclusão**: apaga ou anonimiza os dados pessoais e financeiros, cancela a assinatura no Stripe e mantém apenas o exigido por lei (ex.: registros fiscais de pagamento), com prazo de retenção documentado.
- Lista de suboperadores (Supabase, Vercel, Stripe, provedor de e-mail, Sentry) e menção à transferência internacional de dados (arts. 33 e seguintes).
- Páginas: `/termos`, `/privacidade`, `/cookies`, com **versão e data**. Mudança relevante exige novo aceite.
- Textos legais gerados como **rascunho** e marcados em `LEGAL_REVIEW.md` para revisão por advogado.

11. AVISOS DE RISCO

Aviso geral (rodapé de todas as páginas logadas, landing, simulador e relatórios):

Este aplicativo tem finalidade educacional e de planejamento. As simulações são baseadas em premissas e não representam garantia de rentabilidade. Investimentos envolvem riscos,

inclusive a possibilidade de perda do capital investido. Rentabilidade passada não é garantia de rentabilidade futura. O Patrimônio 1M não é uma instituição financeira e não presta consultoria de valores mobiliários.

Avisos por classe (exibidos onde a classe aparece):

- **Ações:** alta volatilidade; podem perder valor significativo; dividendos não são garantidos.
- **FIIIs:** cotas oscilam; rendimentos variam e podem ser suspensos; risco de vacância, inadimplência e liquidez.
- **Fiagro:** exposição ao agronegócio, a risco de crédito e a riscos climáticos; liquidez pode ser baixa.
- **ETFs internacionais:** risco cambial e de mercado externo; replicação imperfeita do índice.
- **Criptoativos:** volatilidade extrema; risco de perda total; regulação em evolução; risco de custódia.
- **Renda fixa e Tesouro:** risco de crédito do emissor; marcação a mercado pode gerar perdas antes do vencimento; cobertura do FGC só para produtos elegíveis e dentro dos limites.

12. BANCO DE DADOS

PostgreSQL com migrations versionadas, `created_at` e `updated_at` em todas as tabelas, UUID como chave primária e *soft delete* onde fizer sentido. Entregue um diagrama ER em `docs/ERD.md` (Mermaid).

Tabela	Propósito
<code>profiles</code>	1:1 com <code>auth.users</code> ; nome de exibição, faixa etária, perfil de risco, preferências, locale, moeda, <code>role</code>
<code>onboarding_responses</code>	respostas do questionário (versionadas)
<code>consents</code>	tipo, versão do documento, aceite/revogado, data e IP truncado
<code>plans</code> , <code>prices</code>	catálogo espelhando o Stripe (<code>stripe_price_id</code> , intervalo, moeda, ativo)
<code>entitlements</code>	recursos e limites por plano
<code>subscriptions</code>	<code>stripe_customer_id</code> , <code>stripe_subscription_id</code> , status, período, <code>cancel_at_period_end</code>
<code>webhook_events</code>	<code>event_id</code> único, tipo, payload, processado em, erro
<code>coupons</code>	espelho dos cupons e códigos criados via admin
<code>goals</code>	meta, prazo, ativa/arquivada
<code>assumption_sets</code>	premissas de cenário versionadas (taxas, inflação, custos, autor, vigência)
<code>simulations</code>	entradas, <code>assumption_set_id</code> e resultado resumido
<code>model_portfolios</code>	carteiras-modelo do sistema (por classe)
<code>portfolios</code>	carteiras do usuário

Tabela	Propósito
portfolio_targets	alvo por classe (e por ativo na Fase 2)
asset_classes	classes (renda fixa, ações BR, FIIs...) com aviso de risco
assets	catálogo: ticker, nome, classe, mercado, moeda. Sem preços
holdings	posição do usuário por classe/ativo
balance_snapshots	saldo informado pelo usuário por data (fonte do patrimônio no MVP)
transactions	aporte, resgate, provento, compra e venda (Fase 2), com valor, data e classe/ativo
contribution_plans	meta de aporte mensal e crescimento anual
milestones	marcos atingidos e data
asset_prices, asset_fundamentals, corporate_actions	Fase 2: preenchidos só pelo MarketDataProvider, com source e as_of
alerts, notifications	Fase 2: regras e entregas
reports	Fase 2: relatórios gerados
content_articles	central educativa
feature_flags	flags globais e por usuário
audit_logs	ator, ação, alvo, metadados, data
data_requests	pedidos LGPD (exportação e exclusão) e status

Inclua seed com classes, carteiras-modelo, premissas padrão, planos, entitlements, artigos iniciais e um catálogo de ativos de exemplo **sem nenhum preço ou indicador**: ITUB4, BBSE3, TAEF11, PETR4, VALE3, BPAC11, B3SA3; HGLG11, KNRI11, XPML11, KNCR11, KNIP11, HGCR11; WRLD11, NASD11, IVVB11; BTC, ETH. Confirme os dados cadastrais com a fonte oficial na Fase 2, porque tickers podem mudar.

13. ARQUITETURA PARA AS FASES 2 E 3 (interfaces e stubs agora, implementação depois)

13.1 Dados de mercado

```
interface MarketDataProvider {
  getQuote(symbols: string[]): Promise<Quote[]>;
  getHistory(symbol: string, range: DateRange, interval: Interval): Promise<Candle[]>;
  getDistributions(symbol: string, range: DateRange): Promise<Distribution[]>;
  getFundamentals(symbol: string): Promise<Fundamentals>; // P/L, P/VP, ROE, DY, dívida, PL, liquidez
}
```

- Adapters por fornecedor (providers/<nome>), escolhidos por variável de ambiente, e um `MockMarketDataProvider` **usado só em testes**.

- Ingestão por job agendado (cron), cache e registro de `source` e `as_of`. A UI mostra "Dados de em " e o atraso, se houver.
- **Somente APIs licenciadas ou fontes autorizadas.** Proibido scraping. Respeitar os termos de redistribuição de cada fornecedor.

13.2 Motor de Oportunidades (Fase 3, após revisão jurídica)

- O usuário reserva uma parcela da carteira (ex.: 5%) e **define os próprios critérios** a partir de uma lista (queda de X% em N dias, DY acima de Y%, P/VP abaixo de Z, abaixo da média de N meses, classe abaixo do alvo).
- O resultado mostra **quais critérios foram atendidos**, com dados, fonte e data: "Este ativo atende a 3 dos 4 critérios definidos no seu modelo."
- Nunca "compre", "venda", "oportunidade imperdível" ou ranking de "melhores ativos".

13.3 Alertas e notificações

Arquitetura de eventos de domínio (`GoalReached`, `MilestoneReached`, `ContributionOverdue`, `PortfolioDrifted`, `DistributionRecorded`, `PriceTargetHit`), canais plugáveis (no app e e-mail) e preferências por tipo com *opt-out*.

13.4 Relatórios

Relatório mensal: patrimônio inicial e final, aportes, rentabilidade do período (Dietz), proventos, alocação × alvo, desvio da meta, projeção e próximo marco. Geração no servidor e exportação em PDF, sempre com o aviso de risco e as premissas.

13.5 Assistente de IA

- Interface `AIAssistant` com *tools* que **chamam o motor de cálculo e consultam os dados do próprio usuário** (com RLS). A IA não faz contas "de cabeça" nem acessa dados de outros usuários.
- Perguntas-alvo: "Por que minha carteira está desbalanceada?", "Quanto preciso aportar para chegar a R\$ 1 milhão em 15 anos?", "Qual foi meu melhor mês de aportes?", "Quanto registrei de proventos este ano?", "Qual classe está abaixo da meta?"
- Prompt de sistema com as mesmas regras de linguagem e enquadramento: explica, calcula, compara e educa; não recomenda compra ou venda de ativos específicos; cita as premissas.
- Registro de uso para controle de custo e limite por plano.

13.6 Importação de carteira

CSV com template próprio e mapeamento de colunas, cálculo de preço médio e registro de operações e proventos. Nada de credenciais de corretora; integrações só por API oficial ou Open Finance.

14. PAINEL ADMINISTRATIVO (`/admin`)

MVP:

- Usuários (lista, busca, plano, status, bloquear/desbloquear), sem dados financeiros (ver 9.3).
- Assinaturas e cupons (criação via Stripe).
- Preços (criação de novo `Price`, ver seção 8).
- Premissas de simulação (novas versões de `assumption_sets`).
- Carteiras-modelo, catálogo de ativos e artigos.
- Feature flags e logs de auditoria.

Métricas (com definição de cada fórmula exibida em tooltip):

MRR, ARR, usuários totais, Free, Premium, em trial, conversão Free → Premium, novas assinaturas, cancelamentos, churn de logos e de receita, retenção por coorte, ARPU, LTV ($\text{ARPU} \div \text{churn mensal}$) e CAC (**gasto de marketing informado manualmente pelo admin** $\div$ novos pagantes).

15. LANDING PAGE (/)

- **Headline:** Seu primeiro milhão começa com um plano.
- **Subheadline:** Descubra quanto investir, como distribuir seu patrimônio e acompanhe sua evolução até a sua próxima grande meta.
- **CTA:** Começar gratuitamente.
- **Seções:** Como funciona (3 passos) · Calculadora interativa **funcional, sem login** · Carteira-modelo por perfil · Simulações e cenários · Acompanhamento · Planos (Free x Premium) · FAQ (inclui "Vocês recomendam investimentos?" → não, e por quê) · Aviso de risco · CTA final.
- SEO: metadados, Open Graph, `sitemap.xml`, `robots.txt` e dados estruturados.
- Performance: LCP < 2,5 s em 4G e Lighthouse ≥ 90 em todas as categorias.
- Sem depoimentos, números de usuários ou logos de imprensa inventados.

16. ROTAS

Públicas: `/`, `/login`, `/cadastro`, `/recuperar-senha`, `/aprender`, `/aprender/[slug]`, `/planos`, `/termos`, `/privacidade`, `/cookies`

Logadas (MVP): `/onboarding`, `/dashboard`, `/meta`, `/simulador`, `/simulador/comparar`, `/carteira`, `/aportes`, `/configuracoes`, `/configuracoes/privacidade`, `/assinatura`

Checkout: use Stripe Checkout hospedado. `/assinatura/sucesso` e `/assinatura/cancelado` só tratam o retorno. Não criar uma página `/checkout` própria com formulário de cartão.

Admin: `/admin` e subpáginas

Fase 2 (atrás de flag, fora do menu até lançar): `/ativos`, `/fiis`, `/fiagro`, `/exterior`, `/cripto`, `/tesouro`, `/dividendos`, `/relatorios`, `/alertas`

Fase 3: `/oportunidades`, `/assistente`

17. STACK E ESTRUTURA

[DECISÃO] Stack de referência (pode trocar por equivalente, desde que justifique em DECISIONS.md):

- **Frontend:** Next.js (App Router) + React + TypeScript `strict`
- **UI:** Tailwind CSS + shadcn/ui; gráficos com Recharts
- **Backend:** Server Actions e Route Handlers do Next.js + Supabase (Postgres, Auth, RLS, Storage, cron)
- **Pagamentos:** Stripe
- **E-mail:** Resend ou equivalente, com templates em React Email
- **Validação:** Zod · **Decimais:** decimal.js
- **i18n:** next-intl, com `pt-BR` agora e estrutura pronta para `en` e `es`; moeda e locale por usuário
- **Testes:** Vitest (unidade e domínio), testes de RLS contra Supabase local, Playwright (E2E e responsividade)
- **Qualidade:** ESLint, Prettier, Husky com lint-staged, CI no GitHub Actions (lint, typecheck, testes, build)
- **Deploy:** Vercel + Supabase; ambientes `dev`, `staging` e `prod` separados

Módulos (fronteiras claras, sem importações cruzadas indevidas):

```
src/
  app/                # rotas e UI
  components/         # design system
  domain/finance/     # motor de cálculo puro (núcleo)
  modules/
    auth/ billing/ goals/ portfolio/ contributions/
    market-data/ notifications/ reports/ ai/ content/ admin/ privacy/
  lib/                # supabase, stripe, email, rate-limit, logger, i18n
  supabase/migrations/ supabase/seed.sql
  tests/ e2e/
  docs/ (ARCHITECTURE, FINANCE, DECISIONS, LEGAL_REVIEW, ERD, RUNBOOK, SECURITY)
```

18. TESTES E CRITÉRIOS DE ACEITE

Antes de dizer que terminou, **execute e relate**:

- ☐ Testes unitários do motor de cálculo (seção 5.4) passando, com cobertura $\geq 95\%$ em `domain/finance`
- ☐ Testes de RLS: o usuário A não lê nem altera dados do usuário B em nenhuma tabela
- ☐ Testes de permissão: usuário comum não acessa `/admin`; Free não acessa recurso Premium nem pela API
- ☐ E2E: cadastro → confirmação de e-mail → onboarding → diagnóstico → dashboard
- ☐ E2E: recuperação de senha

- ☐ E2E (Stripe em modo de teste): Free → Checkout → Premium; cartão recusado (4000 0000 0000 0341); trial; cancelamento ao fim do período; reativação; downgrade sem perda de dados
 - ☐ Webhooks: assinatura inválida rejeitada; evento duplicado ignorado; eventos fora de ordem tratados
 - ☐ Exclusão de conta: dados removidos ou anonimizados e assinatura cancelada no Stripe
 - ☐ Exportação de dados LGPD gera arquivo completo do usuário
 - ☐ Responsividade com Playwright em 360, 768, 1024 e 1440 px, sem rolagem horizontal
 - ☐ Acessibilidade com axe sem violações críticas
 - ☐ Teste de linguagem proibida (seção 1.3) passando
 - ☐ Nenhum botão ou link sem ação (varredura E2E dos menus)
 - ☐ Build de produção sem warnings de tipo; nenhum segredo no bundle do cliente
-

19. ENTREGÁVEIS

1. Código-fonte organizado conforme a seção 17.
2. `README.md` com setup local em poucos comandos (Supabase local, Stripe CLI, variáveis de ambiente, seed e testes).
3. `.env.example` completo e comentado, sem valores reais.
4. Migrations, seed e script de setup do Stripe.
5. Documentação em `docs/` (arquitetura, fórmulas, decisões, ERD, segurança, runbook e pendências jurídicas).
6. Pipeline de CI configurado.
7. Relatório final: o que foi implementado, o que foi testado (com resultado), o que ficou para as Fases 2 e 3 e os riscos conhecidos.